

The Foundation - System Mission, Ubuntu, Terminal, and GitHub

DR. TNOURJI, ABDELLAH

Research Scientist

The UM6P Vanguard Center

Autonomous Drone Engineering Internship

Mohammed VI Polytechnic University, UM6P , September 8, 2026



University
Mohammed VI
Polytechnic

THE UM6P
VANGUARD
CENTER

#Empowering Minds.

1 Mission and Foundations

1 Mission and Foundations

The Internship Mission

Build a reproducible autonomous-drone inspection system in simulation

Engineering Need

Inspection tasks may involve large, repetitive, difficult-to-access, or potentially hazardous areas. Aerial robots can help collect visual evidence efficiently and consistently.

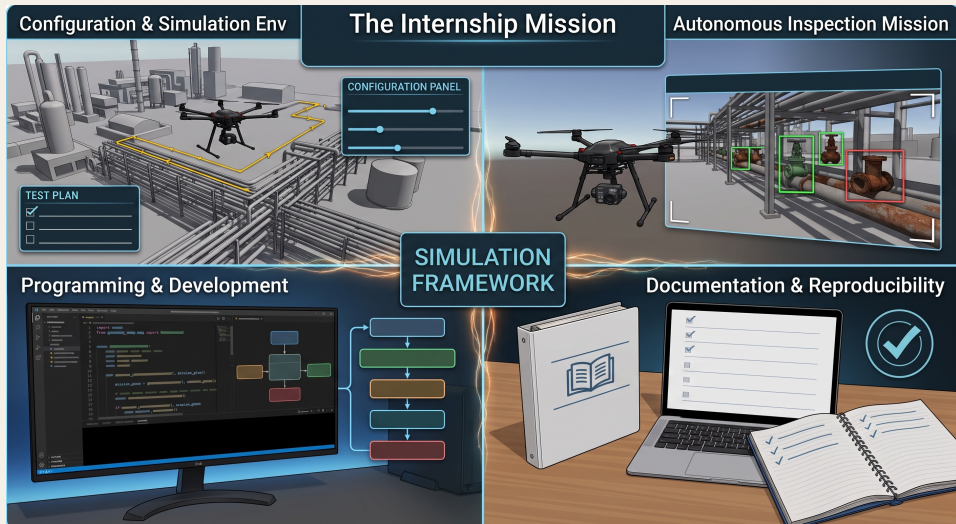
Observe the complete demonstration before studying its components.

Gazebo world → PX4-controlled flight → ROS 2 data exchange → Camera perception →
Mission evidence

Important Boundary

A successful simulation is evidence of software integration, not proof that a real aircraft is ready or safe to fly.

First, Observe the Destination



The Expected Final Demonstration

What the completed GitHub-based project should reproduce

1. Launch

Start the required simulation, flight-control, communication, and perception processes from a documented workspace.

The Expected Final Demonstration

What the completed GitHub-based project should reproduce

1. Launch

Start the required simulation, flight-control, communication, and perception processes from a documented workspace.

2. Observe

Watch the vehicle, mission state, camera stream, ROS 2 interfaces, and detector outputs.

The Expected Final Demonstration

What the completed GitHub-based project should reproduce

1. Launch

Start the required simulation, flight-control, communication, and perception processes from a documented workspace.

2. Observe

Watch the vehicle, mission state, camera stream, ROS 2 interfaces, and detector outputs.

3. Verify

Compare intended behavior with logs, messages, detections, screenshots, and saved mission evidence.

Definition of Success

The system launches predictably, its components exchange the expected information, the simulated drone executes the intended mission, and the results can be explained and reproduced from the repository documentation.

The Engineering Challenge

One visible mission depends on several connected technical layers

Why Is the Project Technically Interesting?

A drone inspection mission is a **system-of-systems problem**. Motion, sensing, communication, software, perception, testing, and documentation must work together.

Operating environment → Simulation → Flight control → Communication → Perception → Evidence

The Engineering Challenge

One visible mission depends on several connected technical layers

Why Is the Project Technically Interesting?

A drone inspection mission is a **system-of-systems problem**. Motion, sensing, communication, software, perception, testing, and documentation must work together.

Operating environment → Simulation → Flight control → Communication → Perception → Evidence

A Chain Is Verified Link by Link

A detector cannot process a missing camera stream. A mission node cannot guide the vehicle through an incorrect interface. A working local setup cannot be reproduced without controlled dependencies and documentation.

Central Principle

Do not memorize isolated commands. Understand each component's responsibility, its inputs and outputs, its connections, and the observation that proves it works.

The Student Journey

Progress from guided reproduction to engineering ownership

1. **Orient:** understand the mission, architecture, repository, and expected evidence.
2. **Prepare:** configure Ubuntu, tools, dependencies, workspace, and project files.
3. **Learn the foundations:** practise Bash, Git, Python, ROS 2, Gazebo, PX4, and perception concepts in context.
4. **Integrate:** connect mission logic, simulated flight, camera data, and object detection.
5. **Verify:** test one layer at a time before testing the complete pipeline.
6. **Deliver:** publish reproducible code, configuration, evidence, documentation, and a technical report.

Day 1 Perspective

You are not expected to master every technology immediately. First build a reliable mental map; then develop each branch through focused tutorials and practical work.

Technology Ecosystem I

The engineering environment and reproducible workflow

Ubuntu and Linux

Ubuntu is the Linux distribution used as the project environment. It provides the filesystem, processes, users, packages, and services on which the robotics tools run.

Terminal and Bash

The **terminal** is the text-based interface window. **Bash** is the shell that interprets commands, variables, scripts, and environment configuration entered through it.

Git

Git records project history locally. Students use it to inspect changes, create commits, recover earlier states, and work systematically.

GitHub

GitHub hosts and shares the Git repository. It supports collaboration, documentation, issue tracking, review, and delivery of the reproducible internship project.

Project Connection

These foundations let students install the environment, obtain the repository, run documented commands, diagnose problems, preserve changes, and share verified results.

Technology Ecosystem II

Programming, integration, simulation, control, and perception

Technology	Intuitive Role	Internship Use
Python	Practical programming language for readable robotics logic and data processing	ROS 2 nodes, mission utilities, image handling, AI integration, and tests
ROS 2	Communication and integration framework, not an operating system	Connect nodes through typed topics, messages, services, parameters, and packages
Gazebo	Simulated world containing the drone, sensors, objects, environment, and physics	Develop and observe the mission without claiming real-world flight readiness
PX4	Autopilot responsible for estimation, stabilization, and tracking flight references	Convert mission references into controlled simulated vehicle motion
AI / YOLOv8	Perception component that transforms camera images into object detections	Produce observations for mission evidence; it does not directly guarantee safe control

Responsibility Boundary

Object detection answers, “What may be visible in this image?” Autonomous control answers, “How should the vehicle move safely?” These are connected responsibilities, not the same function.

Complete System Pipeline

From engineering environment to reproducible mission evidence

Foundation and Execution Layer

Ubuntu + Terminal/Bash + Python environment $\longrightarrow$ workspace, dependencies, scripts, and running processes

Complete System Pipeline

From engineering environment to reproducible mission evidence

Foundation and Execution Layer

Ubuntu + Terminal/Bash + Python environment $\longrightarrow$ workspace, dependencies, scripts, and running processes

Command Flow: Mission Decision to Simulated Motion

ROS 2 mission node → Micro XRCE-DDS → PX4 autopilot → Gazebo drone physics

Complete System Pipeline

From engineering environment to reproducible mission evidence

Foundation and Execution Layer

Ubuntu + Terminal/Bash + Python environment → workspace, dependencies, scripts, and running processes

Command Flow: Mission Decision to Simulated Motion

ROS 2 mission node → Micro XRCE-DDS → PX4 autopilot → Gazebo drone physics

Evidence Flow: Simulated World to Inspection Result

Simulated camera → ROS-Gazebo image bridge → YOLOv8 detector → mission evidence and report

Complete System Pipeline

From engineering environment to reproducible mission evidence

Foundation and Execution Layer

Ubuntu + Terminal/Bash + Python environment → workspace, dependencies, scripts, and running processes

Command Flow: Mission Decision to Simulated Motion

ROS 2 mission node → Micro XRCE-DDS → PX4 autopilot → Gazebo drone physics

Evidence Flow: Simulated World to Inspection Result

Simulated camera → ROS-Gazebo image bridge → YOLOv8 detector → mission evidence and report

Reproducibility Layer

Git and GitHub preserve the source code, configuration, history, instructions, tests, limitations, and evidence required to rebuild the complete pipeline.

How We Will Study Each Foundation

A repeated reasoning pattern for every tutorial

1. **Why?** Which project problem does this foundation solve?
2. **What?** What does it mean at an intuitive and technically correct level?
3. **How does it fit?** Where is it located in the complete pipeline?
4. **Project connection?** Which file, process, command, node, topic, or result uses it?
5. **Verification?** What observable output confirms correct behavior?
6. **Next step?** Which later tutorial develops the idea in greater depth?

Example

Before studying ROS 2 details, students first recognize its system role: it carries typed information between software components and makes their interfaces observable and testable.

Professional Working Method

Progress is created through disciplined observation

Before Execution

- ▶ identify the current directory and environment;
- ▶ predict the expected output;
- ▶ understand which component will be affected;
- ▶ change one factor at a time.

After Execution

- ▶ read complete outputs and error messages;
- ▶ compare observation with expectation;
- ▶ preserve commands, logs, and screenshots;
- ▶ document results and limitations honestly.

Engineering Habit

A desired trajectory, a clean visualization, or a single successful launch is not sufficient evidence. Each important claim must be supported by an observable test.

Internship Deliverables

What students are expected to build and submit

Working Engineering Assets

- ▶ structured Git repository and clear history;
- ▶ reproducible environment and configuration;
- ▶ ROS 2 packages, Python source code, and launch files;
- ▶ working Gazebo/PX4 inspection simulation;
- ▶ camera and object-detection integration.

Evidence and Communication

- ▶ layer-level and end-to-end tests;
- ▶ logs, screenshots, and mission results;
- ▶ installation and execution instructions;
- ▶ final demonstration;
- ▶ technical report with results, limitations, and future work.

Delivery Standard

A reviewer should be able to clone the repository, follow the documentation, reproduce the demonstration, and understand what was verified.

Knowledge Check

Can you explain the project before implementing it?

Mission and Architecture

1. What useful inspection outcome should the system produce?
2. Why is this a systems-engineering project?
3. Which components simulate physics, stabilize flight, and exchange information?

Verification and Boundaries

1. Why verify the camera stream before the detector?
2. Why is detection different from control?
3. Why does simulation success not prove real-flight readiness?

Expected Explanation

Describe the mission, command flow, evidence flow, role of each major technology, and the evidence required to reproduce the result.

Learning Roadmap

Each foundation unlocks one part of the final GitHub project

Foundation	Contribution to the Project	Practical Outcome
Ubuntu	Provides the Linux engineering environment	Prepare the workstation
Terminal and Bash	Operate, configure, launch, inspect, and troubleshoot	Run the documented workflow
Git and GitHub	Preserve and share a reproducible project	Clone, manage, document, and deliver
Python for robotics	Express logic and process project data	Read and modify project programs
ROS 2	Connect software components through defined interfaces	Build and verify communication
Gazebo and PX4	Simulate the world and controlled drone motion	Execute the inspection mission
AI integration	Convert camera data into perception results	Produce inspection evidence

End-to-End Objective

The tutorials are not isolated courses. Together, they provide the foundations required to reproduce,

Transition to the Technical Foundations

Build the environment before building the autonomous system

Next: Ubuntu and the Linux Engineering Environment

Ubuntu → Terminal and Bash → Git and GitHub → Python → ROS 2 → Gazebo and PX4 → AI integration

Guiding Question

What must be installed, organized, and verified so that every later component can run reliably inside the same engineering environment?